

WEEK 1: SECURITY FOUNDATIONS REFERENCE PACK

Cybersecurity Internship • Security-Foundations-Reference-Pack

Purpose: This document provides the study notes, practical exercises, evidence checklist, and reporting templates required for Week 1. It is designed to support the four Week 1 tasks: Networking Fundamentals, Linux & System Security Exercises, Threat & Vulnerability Identification, and Basic Security Assessment.

1. Week 1 Deliverables

Task	Main Activities	Expected Evidence	Status
1.1 Networking Fundamentals	Study TCP/IP, DNS, HTTP and common ports; create notes and a diagram.	Notes document, network diagram, screenshots if applicable.	To complete
1.2 Linux & System Security	Practice CLI, permissions, users and basic hardening in an authorized lab.	Terminal logs, screenshots and explanatory notes.	To complete
1.3 Threat & Vulnerability Identification	Research common web-system threats, vulnerabilities and attack vectors.	Threat/vulnerability reference document.	To complete
1.4 Basic Security Assessment	Assess a local/controlled VM or system using safe, non-destructive checks.	Assessment notes, commands, findings and screenshots.	To complete

2. Task 1 — Networking Fundamentals

TCP/IP Overview

TCP/IP is the collection of protocols used to allow devices and applications to communicate across networks. The model is commonly described using four layers:

- Application layer: services used by applications, such as HTTP, DNS, SSH and SMTP.
- Transport layer: end-to-end communication using TCP or UDP. TCP provides reliability and ordering; UDP has lower overhead and does not guarantee delivery.
- Internet layer: logical addressing and routing using IP. IPv4 and IPv6 operate here.
- Network Access layer: communication over the local network, including Ethernet and Wi-Fi.

2.2 Important Networking Concepts

- IP address: identifies a network interface on an IP network.
- MAC address: hardware/link-layer address used on a local network.
- Subnet mask/prefix: determines the network and host portions of an IP address.
- Default gateway: router used to reach networks outside the local subnet.
- Router: forwards traffic between networks.
- Switch: connects devices within a local network.
- Port: logical endpoint used by network services.
- Protocol: defined set of rules for communication.

DNS

The Domain Name System translates human-readable domain names into IP addresses and can also provide other records. Common records include:

- A: maps a hostname to an IPv4 address.
- AAAA: maps a hostname to an IPv6 address.
- CNAME: creates an alias for another hostname.
- MX: identifies mail servers for a domain.
- TXT: stores text data, often used for verification and email security policies.
- NS: identifies authoritative name servers.

Security considerations: DNS spoofing, cache poisoning, unauthorized zone transfers, and misleading look-alike domains.

HTTP and HTTPS

HTTP is an application-layer protocol used to transfer web requests and responses. HTTPS is HTTP protected by TLS, which helps provide confidentiality, integrity and server authentication.

Method/Status	Meaning	Security Relevance
GET	Retrieve a resource	Should not normally change server state.
POST	Submit data or trigger processing	Requires input validation and authorization checks.
PUT/PATCH	Create or update a resource	Must enforce authorization and validation.
DELETE	Remove a resource	Must be protected by strong authorization.
200	Successful request	Check whether sensitive data is exposed.
401	Authentication required/failed	Review authentication handling.
403	Forbidden	Review authorization and information disclosure.
404	Resource not found	Avoid excessive technical details.
500	Server error	Avoid stack traces and sensitive error messages.

Common Ports

Port	Service	Security Notes
20/21	FTP	Plain FTP can expose credentials and data; prefer secure alternatives.
22	SSH	Use keys, strong authentication, patching and restricted access.
23	Telnet	Unencrypted and generally unsafe.
25	SMTP	Mail transfer; configuration and abuse controls matter.
53	DNS	Can use UDP/TCP; protect DNS infrastructure.
80	HTTP	Unencrypted web traffic.

110	POP3	Email retrieval; use encrypted variants.
143	IMAP	Email access; use TLS.
443	HTTPS	Encrypted web traffic; TLS must be configured correctly.
3306	MySQL	Do not expose database services unnecessarily.
3389	RDP	Restrict access, use MFA/VPN where appropriate and monitor attempts.

Suggested Networking Commands

- `ip addr` — Displays network interfaces and IP addresses.
- `ip route` — Displays routing information.
- `ping 127.0.0.1` — Tests the local TCP/IP stack.
- `ping <gateway-ip>` — Tests reachability to the local gateway.
- `nslookup example.com` — Queries DNS information.
- `curl -I https://example.com` — Displays HTTP response headers.
- `ss -tln` — Shows listening TCP/UDP sockets.
- `traceroute example.com` — Shows the path toward a destination where supported.

Evidence placeholder: Insert a network diagram showing your computer/VM, router or gateway, DNS, web service and the direction of traffic.

Task 2 — Linux & System Security Exercises

Perform these exercises only in your own Ubuntu/Linux virtual machine or another system for which you have explicit authorization. Record the command, purpose, result and screenshot.

System and Identity Checks

- `uname -a` — displays kernel and system information.
- `cat /etc/os-release` — identifies the Linux distribution and version.
- `whoami` — displays the current user.
- `id` — displays user ID, group ID and group memberships.
- `hostnamectl` — displays or manages hostname information where supported.

Files, Permissions and Ownership

- `ls -la` — lists files, including hidden files, with permissions and ownership.
- `pwd` — displays the current working directory.
- `mkdir security-lab` — creates a practice directory.
- `touch test.txt` — creates an empty file.
- `chmod 600 test.txt` — allows the owner to read and write the file.
- `chmod 644 test.txt` — allows the owner to read/write and others to read.
- `chown <user>:<group> test.txt` — changes ownership when authorized.

Permission notation example: `-rwxr-x---` means the owner has read/write/execute, the group has read/execute, and others have no permissions. Avoid granting broad permissions such as `777` unless there is a controlled and documented reason.

User and Privilege Management

- `getent passwd` — lists local and configured user account information.
- `getent group` — lists group information.
- `groups` — displays the groups associated with the current user.
- `sudo -l` — shows commands the current user may run with sudo, subject to policy.
- `passwd` — changes the current user's password.
- `sudo adduser labuser` — creates a practice user when appropriate and authorized.

Security principles: least privilege, unique accounts, strong authentication, removal of unused accounts, and careful control of sudo access.

Basic Hardening Checklist

- Apply operating-system updates in the lab.
- Use strong passwords and avoid sharing accounts.
- Disable or remove unnecessary services.
- Restrict exposed ports using a host firewall.
- Use SSH keys where SSH is required and disable insecure authentication methods when appropriate.
- Review user accounts and privileged groups.
- Avoid storing secrets in scripts, screenshots or Git repositories.
- Enable logging and review authentication events.
- Use backups or snapshots before making configuration changes.

Firewall and Listening-Service Review

- `sudo ufw status verbose` — checks UFW status and rules if UFW is installed.
- `ss -tln` — identifies listening sockets.
- `systemctl --type=service --state=running` — lists running services on systems using systemd.
- `sudo journalctl -p warning -b` — reviews warning-level events from the current boot, where supported.

Do not disable services or change firewall rules blindly. First identify the service, its purpose and the possible impact.

4. Task 3 — Threat & Vulnerability Identification

A threat is a potential cause of harm. A vulnerability is a weakness that could be exploited. Risk is commonly considered as a combination of likelihood and impact. An attack vector is the path or method used to reach a target.

Threat/Vulnerability	Example	Potential Impact	Basic Mitigation
Broken access control	A user accesses another user's record by changing an ID.	Unauthorized disclosure or modification.	Server-side authorization checks; deny by default; test object-level access.
Injection	Untrusted input is interpreted as a command or query.	Data exposure, modification or system compromise.	Parameterized queries, safe APIs, validation and least privilege.
Cross-site scripting (XSS)	User input is rendered as an executable browser script.	Session abuse, data theft or content manipulation.	Output encoding, input handling, CSP and secure cookies.

Authentication weakness	Weak passwords or missing rate limits.	Account takeover.	MFA, secure password storage, rate limiting and monitoring.
Security misconfiguration	Debug mode or default credentials remain enabled.	Information disclosure or unauthorized access.	Secure defaults, hardening, patching and configuration review.
Sensitive data exposure	Secrets or personal data appear in logs or responses.	Privacy breach and credential compromise.	Minimize data, encrypt appropriately, redact logs and restrict access.
Vulnerable components	An outdated framework contains a known flaw.	Exploitation of the vulnerable component.	Inventory dependencies, patch and remove unused components.
Insecure design	A workflow lacks abuse-case analysis.	Business logic abuse or fraud.	Threat modeling, secure design reviews and abuse-case testing.
SSRF	A server fetches attacker-controlled URLs.	Access to internal services or metadata.	Allow-list destinations, network segmentation and response filtering.
Denial of service	Excessive requests exhaust resources.	Service unavailability.	Rate limits, resource controls, caching and monitoring.

Threat Identification Questions

- What assets need protection?
- Who should be allowed to access each asset?
- What inputs are controlled by users or external systems?
- What happens if authentication or authorization fails?
- What sensitive information is stored, transmitted or logged?
- Which services, ports, dependencies and administrative interfaces are exposed?
- What would be the impact if this weakness were exploited?
- What control could prevent, detect or reduce the impact?